```
<!-- src/app/components/upload/upload.component.html -->

<div class="upload-container" (drop)="onDrop($event)" (dragover)="onDragOver($event)"
(dragenter)="onDragEnter($event)" (dragleave)="onDragLeave($event)">

 <h3>Subir Audio para Transcripción</h3>

 <div class="upload-area" [class.drag-active]="isDragging">

  <input

   type="file"

   id="fileInput"

   accept="audio/*"

   (change)="onFileSelected($event)"

   [disabled]="uploading"

   style="display: none;"

  />

  <div *ngIf="!selectedFile && !uploading" class="upload-prompt" (click)="triggerFileInput()">

   <svg xmlns="http://www.w3.org/2000/svg" width="48" height="48" viewBox="0 0 24 24"
fill="none" stroke="currentColor"

    stroke-width="2">

    <path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4"></path>

    <polyline points="17 8 12 3 7 8"></polyline>

    <line x1="12" y1="3" x2="12" y2="15"></line>

   </svg>

   <p>Haz clic o arrastra un archivo aquí para seleccionar un archivo de audio</p>

   <p class="upload-hint">Formatos soportados: MP3, WAV, MP4, OGG</p>

  </div>

  <div *ngIf="selectedFile && !uploading" class="file-info">

   <p><strong>Archivo seleccionado:</strong></p>

   <p>{{ selectedFile.name }}</p>

   <div class="file-size-info">

    <p>Tamaño del archivo: {{ getFileSizeLabel(selectedFile.size) }}</p>

    <div class="progress-bar">

     <div class="progress" [style.width.%]="getFileProgress(selectedFile.size)"></div>

    </div>

   </div>

   <div class="button-group">

    <button

     type="button"

     class="btn btn-primary"

     (click)="uploadFile()"

     [disabled]="uploading"

    >

     Subir y Transcribir

    </button>

    <button

     type="button"

     class="btn btn-secondary"

     (click)="clearSelection()"

     [disabled]="uploading"

    >

     Cancelar

    </button>

   </div>

  </div>

  <div *ngIf="uploading" class="uploading">

   <div class="spinner"></div>

   <p>Subiendo archivo...</p>

  </div>

  <div *ngIf="errorMessage" class="alert alert-error">

   {{ errorMessage }}

  </div>

  <div *ngIf="successMessage" class="alert alert-success">

   {{ successMessage }}

  </div>

 </div>

 /* src/app/components/upload/upload.component.scss */

 .upload-container {

  background: white;

  border-radius: 12px;

  padding: 24px;

  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);


  h3 {

   margin-bottom: 20px;

   color: #2c3e50;

   font-size: 20px;

  }

 }


 .upload-area {

  border: 2px dashed #cbd5e0;

  border-radius: 8px;

  padding: 40px;

  text-align: center;

  transition: all 0.3s ease;

  min-height: 200px;

  display: flex;

  flex-direction: column;

  align-items: center;

  justify-content: center;


  &:hover {

   border-color: #3498db;

   background: #f8fafc;

  }


  input[type="file"] {

   position: absolute;

   width: 100%;

   height: 100%;

   top: 0;

   left: 0;

   opacity: 0;

   cursor: pointer;
```

```
  }
 }

 .upload-prompt {
  svg {
   color: #3498db;
   margin-bottom: 16px;
  }

  p {
   font-size: 16px;
   color: #4a5568;
   margin: 8px 0;
  }
 }

 .upload-hint {
  font-size: 13px;
  color: #a0aec0;
 }

 .file-info {
  text-align: center;

  p {
   margin: 8px 0;
   font-size: 14px;
  }
 }

 .file-size {
  color: #718096;
  font-size: 13px;
 }

 .button-group {
  display: flex;
  gap: 12px;
  justify-content: center;
  margin-top: 20px;
 }

 .uploading {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 16px;
 }

 .drag-active {
  border-color: #3498db;
  background-color: #f0f8ff;
 }

 .file-size-info {
  margin-top: 10px;
 }

 .progress-bar {
  height: 10px;
  background-color: #e0e0e0;
  border-radius: 5px;
  overflow: hidden;
 }

 .progress {
  height: 100%;
  background-color: #3498db;
  transition: width 0.3s ease;
 }
 // src/app/components/upload/upload.component.ts - CORREGIDO COMPLETO
 // src/app/components/upload/upload.component.ts
 import { Component, EventEmitter, Output } from '@angular/core';
 import { CommonModule } from '@angular/common';
 import { AudioService } from '../../services/audio.service';
 import { environment } from '../../../environments/environment';

 @Component({
  selector: 'app-upload',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './upload.component.html',
  styleUrls: ['./upload.component.scss']
 })
 export class UploadComponent {
  @Output() uploadComplete = new EventEmitter<void>();

  selectedFile: File | null = null;
  uploading = false;
  uploadProgress = 0;
  errorMessage = '';
  successMessage = '';
  maxFileSize = environment.maxFileSize;
  isDragging = false;

  constructor(private audioService: AudioService) {
   console.log('UploadComponent inicializado');
  }

  triggerFileInput(): void {
   const fileInput = document.getElementById('fileInput') as HTMLInputElement;
   if (fileInput) {
```

```
    fileInput.click();
  }
 }

 onFileSelected(event: Event): void {
  const input = event.target as HTMLInputElement;
  if (input.files && input.files.length > 0) {
   const file = input.files[0];
   console.log('Archivo seleccionado:', {
    name: file.name,
    size: file.size,
    type: file.type
   });
   if (file.size > this.maxFileSize) {
    this.errorMessage = `El archivo excede el tamaño máximo permitido (${this.maxFileSize / 1024 / 1024} MB)`;
    this.selectedFile = null;
    console.error('Archivo demasiado grande');
    return;
   }
   this.selectedFile = file;
   this.errorMessage = '';
   this.successMessage = '';
   console.log('Archivo listo para subir');
  }
 }

 uploadFile(): void {
  console.log('uploadFile() LLAMADO');
  console.log('selectedFile:', this.selectedFile);
  if (!this.selectedFile) {
   this.errorMessage = 'Por favor selecciona un archivo';
   console.error('No hay archivo seleccionado');
   return;
  }
  this.uploading = true;
  this.uploadProgress = 0;
  this.errorMessage = '';
  this.successMessage = '';

  console.log('Iniciando subida al servidor:', this.selectedFile.name);
  this.audioService.uploadAudio(this.selectedFile).subscribe({
   next: (response) => {
    console.log('Respuesta del servidor:', response);
    this.uploading = false;
    this.uploadProgress = 100;
    this.successMessage = response.message || 'Audio subido exitosamente';
    const tempFileName = this.selectedFile?.name;
    this.selectedFile = null;
    const fileInput = document.getElementById('fileInput') as HTMLInputElement;
    if (fileInput) {
     fileInput.value = '';
    }
    console.log('Archivo subido exitosamente:', tempFileName);
    setTimeout(() => {
     this.uploadComplete.emit();
     this.successMessage = '';
     console.log('Upload complete event emitido');
    }, 1500);
   },
   error: (error) => {
    console.error('ERROR COMPLETO:', error);
    console.error('Error response:', error.error);
    console.error('Error status:', error.status);
    console.error('Error message:', error.message);
    this.uploading = false;
    this.uploadProgress = 0;
    let errorMsg = 'Error al subir el archivo';
    if (error.error?.message) {
     errorMsg = error.error.message;
    } else if (error.message) {
     errorMsg = error.message;
    }
    this.errorMessage = errorMsg;
    console.error('Error mostrado al usuario:', this.errorMessage);
   },
   complete: () => {
    console.log('Upload completado o finalizado');
   }
  });
 }

 clearSelection(): void {
  console.log('Limpiando selección');
  this.selectedFile = null;
  this.errorMessage = '';
  this.successMessage = '';
  const fileInput = document.getElementById('fileInput') as HTMLInputElement;
  if (fileInput) fileInput.value = '';
 }

 onDragOver(event: DragEvent): void {
  event.preventDefault();
  event.stopPropagation();
  this.isDragging = true;
 }

 onDragEnter(event: DragEvent): void {
  event.preventDefault();
  event.stopPropagation();
  this.isDragging = true;
 }
```

```
  onDragLeave(event: DragEvent): void {
    event.preventDefault();
    event.stopPropagation();
    this.isDragging = false;
  }

  onDrop(event: DragEvent): void {
    event.preventDefault();
    event.stopPropagation();
    this.isDragging = false;

    const files = event.dataTransfer?.files;
    if (files && files.length > 0) {
      const eventObj = { target: { files } } as unknown as Event;
      this.onFileSelected(eventObj);
    }
  }

  getFileSizeLabel(size: number): string {
    if (size < 1024) {
      return size + ' bytes';
    } else if (size < 1024 * 1024) {
      return (size / 1024).toFixed(2) + ' KB';
    } else {
      return (size / (1024 * 1024)).toFixed(2) + ' MB';
    }
  }

  getFileProgress(size: number): number {
    const maxSize = this.maxFileSize;
    return (size / maxSize) * 100;
  }
}
// src/controllers/audio.controller.ts - CORREGIDO COMPLETO
import { Response, NextFunction } from 'express';
import { StatusCodes } from 'http-status-codes';
import { PrismaClient, AudioStatus } from '@prisma/client';
import { AuthRequest } from '../middleware/auth.middleware';
import { AppError } from '../middleware/error.middleware';
import driveService from '../services/drive.service';
import queueService from '../services/queue.service';
import logger from '../utils/logger';
import { config } from '../config/env';

const prisma = new PrismaClient();

export class AudioController {
  async uploadAudio(req: AuthRequest, res: Response, next: NextFunction): Promise<void> {
    try {
      logger.info('=== INICIO UPLOAD AUDIO ===');

      const userId = req.user!.id;
      const file = req.file;

      logger.info('Usuario:', { userId });
      logger.info('Archivo recibido:', {
        exists: !!file,
        name: file?.originalname,
        size: file?.size,
        mimetype: file?.mimetype
      });

      if (!file) {
        logger.error('No se recibió ningún archivo');
        throw new AppError('No se proporcionó ningún archivo', StatusCodes.BAD_REQUEST);
      }

      // Verificar cuota diaria
      const today = new Date();
      today.setHours(0, 0, 0, 0);
      const tomorrow = new Date(today);
      tomorrow.setDate(tomorrow.getDate() + 1);

      let quota = await prisma.uploadQuota.findFirst({
        where: {
          userId,
          date: {
            gte: today,
            lt: tomorrow
          }
        }
      });

      if (!quota) {
        try {
          quota = await prisma.uploadQuota.create({
            data: {
              userId,
              date: today,
              count: 0,
            },
          });
          logger.info('Cuota creada para usuario');
        } catch (createError: any) {
          logger.warn('Error al crear cuota, buscando de nuevo...', { error: createError.message });
          quota = await prisma.uploadQuota.findFirst({
            where: {
              userId,
              date: {
                gte: today,
                lt: tomorrow
              }
```

```
        }
       });
       if (!quota) {
        logger.error('No se pudo crear ni encontrar la cuota');
        throw new AppError('Error al verificar cuota diaria',
StatusCodes.INTERNAL_SERVER_ERROR);
       }
      }
     }

     logger.info('Cuota actual:', {
      quotaId: quota.id,
      count: quota.count,
      limit: config.limits.maxDailyUploads
     });

     if (quota.count >= config.limits.maxDailyUploads) {
      logger.warn('Límite diario alcanzado');
      throw new AppError(
       `Límite diario alcanzado (${config.limits.maxDailyUploads} archivos)`,
       StatusCodes.TOO_MANY_REQUESTS
      );
     }

     // Subir a Google Drive
     const timestamp = Date.now();
     const filename = `audio_${userId}_${timestamp}_${file.originalname}`;
     logger.info('Subiendo a Google Drive...', { filename });

     let driveResult;
     try {
      driveResult = await driveService.uploadFile(
       file.buffer,
       filename,
       file.mimetype
      );
      logger.info('Archivo subido a Drive', driveResult);
     } catch (driveError: any) {
      logger.error('Error al subir a Drive:', { error: driveError.message });
      throw new AppError('Error al subir archivo a Google Drive',
StatusCodes.INTERNAL_SERVER_ERROR);
     }

     const { fileId, webViewLink } = driveResult;

     // Usar transacción
     const result = await prisma.$transaction(async (tx) => {
      const audio = await tx.audio.create({
       data: {
        userId,
        filename,
        originalFilename: file.originalname,
        driveFileId: fileId,
        driveFileUrl: webViewLink,
        status: AudioStatus.PENDING,
        fileSize: file.size,
        mimeType: file.mimetype,
       },
      });

      await tx.uploadQuota.update({
       where: { id: quota!.id },
       data: {
        count: quota!.count + 1
       },
      });

      return audio;
     });

     logger.info('Registro creado en BD y cuota actualizada', { audioId: result.id });

     // Encolar
     try {
      await queueService.addAudioJob({
       audioId: result.id,
       userId,
       driveFileId: fileId,
       driveFileUrl: webViewLink,
       filename,
      });
      logger.info('Audio encolado para procesamiento');
     } catch (queueError: any) {
      logger.error('Error al encolar audio:', { error: queueError.message });
     }

     logger.info('=== FIN UPLOAD AUDIO EXITOSO ===');

     res.status(StatusCodes.CREATED).json({
      success: true,
      message: 'Audio subido exitosamente y encolado para transcripción',
      data: {
       id: result.id,
       filename: result.originalFilename,
       status: result.status,
       createdAt: result.createdAt,
      },
     });
    } catch (error: any) {
     logger.error('=== ERROR EN UPLOAD AUDIO ===', {
      message: error.message,
      code: error.code,
      stack: error.stack
```

```
    });
    next(error);
  }
 }

// ========== MÉTODO CORREGIDO: getUserAudios ==========
async getUserAudios(req: AuthRequest, res: Response, next: NextFunction): Promise<void> {
  try {
    const userId = req.user!.id;
    const { status, startDate, endDate, limit = 50, offset = 0 } = req.query;

    const where: any = { userId };
    if (status) {
      where.status = status as AudioStatus;
    }
    if (startDate && endDate) {
      const start = new Date(startDate as string);
      const end = new Date(endDate as string);
      end.setDate(end.getDate() + 1); // Ajustar la fecha de finalización al final del día

      where.createdAt = {
        gte: start,
        lt: end,
      };
    }

    const [audios, total] = await Promise.all([
      prisma.audio.findMany({
        where,
        orderBy: { createdAt: 'desc' },
        take: Number(limit),
        skip: Number(offset),
        select: {
          id: true,
          filename: true,
          originalFilename: true,
          status: true,
          transcriptionText: true,

          // ========== AGREGAR ESTOS DOS CAMPOS ==========
          driveTranscriptionFileId: true,
          driveTranscriptionFileUrl: true,
          // ==============================================

          errorMessage: true,
          fileSize: true,
          duration: true,
          createdAt: true,
          updatedAt: true,
          processingStarted: true,
          processingFinished: true,
        },
      }),
      prisma.audio.count({ where }),
    ]);

    res.json({
      success: true,
      data: audios,
      pagination: {
        total,
        limit: Number(limit),
        offset: Number(offset),
      },
    });
  } catch (error) {
    next(error);
  }
}

// ========== MÉTODO CORREGIDO: getAllAudios ==========
async getAllAudios(req: AuthRequest, res: Response, next: NextFunction): Promise<void> {
  try {
    const { status, userId, startDate, endDate, limit = 50, offset = 0 } = req.query;

    const where: any = {};
    if (status) where.status = status as AudioStatus;
    if (userId) where.userId = userId as string;
    if (startDate && endDate) {
      where.createdAt = {
        gte: new Date(startDate as string),
        lte: new Date(endDate as string),
      };
    }
    const [audios, total] = await Promise.all([
      prisma.audio.findMany({
        where,
        orderBy: { createdAt: 'desc' },
        take: Number(limit),
        skip: Number(offset),
        select: {
          id: true,
          filename: true,
          originalFilename: true,
          status: true,
          transcriptionText: true,

          // ========== AGREGAR ESTOS DOS CAMPOS ==========
          driveTranscriptionFileId: true,
          driveTranscriptionFileUrl: true,
          // ==============================================
```

```
        errorMessage: true,
        fileSize: true,
        duration: true,
        createdAt: true,
        updatedAt: true,
        processingStarted: true,
        processingFinished: true,
        userId: true, // Para admin
      },
    }),
    prisma.audio.count({ where }),
  ]);

  res.json({
    success: true,
    data: audios,
    pagination: {
      total,
      limit: Number(limit),
      offset: Number(offset),
    },
  });
 } catch (error) {
  next(error);
 }
}

async getAudioById(req: AuthRequest, res: Response, next: NextFunction): Promise<void> {
 try {
  const { id } = req.params;
  const userId = req.user!.id;
  const isAdmin = req.user!.role === 'admin';

  const audio = await prisma.audio.findUnique({
   where: { id },
  });

  if (!audio) {
   throw new AppError('Audio no encontrado', StatusCodes.NOT_FOUND);
  }

  if (!isAdmin && audio.userId !== userId) {
   throw new AppError('No tienes permiso para ver este audio', StatusCodes.FORBIDDEN);
  }

  res.json({
   success: true,
   data: audio,
  });
 } catch (error) {
  next(error);
 }
}

async deleteAudio(req: AuthRequest, res: Response, next: NextFunction): Promise<void> {
 try {
  const { id } = req.params;
  const userId = req.user!.id;
  const isAdmin = req.user!.role === 'admin';

  const audio = await prisma.audio.findUnique({
   where: { id },
  });

  if (!audio) {
   throw new AppError('Audio no encontrado', StatusCodes.NOT_FOUND);
  }

  if (!isAdmin && audio.userId !== userId) {
   throw new AppError('No tienes permiso para eliminar este audio',
StatusCodes.FORBIDDEN);
  }

  if (audio.driveFileId) {
   try {
    await driveService.deleteFile(audio.driveFileId);
   } catch (error) {
    logger.warn('Error al eliminar archivo de Drive', { error });
   }
  }

  await queueService.removeJob(id);
  await prisma.audio.delete({
   where: { id },
  });

  logger.info('Audio eliminado exitosamente', { audioId: id, userId });

  res.json({
   success: true,
   message: 'Audio eliminado exitosamente',
  });
 } catch (error) {
  next(error);
 }
}

async getStats(req: AuthRequest, res: Response, next: NextFunction): Promise<void> {
 try {
  const userId = req.user!.id;
  const isAdmin = req.user!.role === 'admin';
  const where: any = isAdmin ? {} : { userId };
```

```
    const [total, pending, processing, done, error] = await Promise.all([
      prisma.audio.count({ where }),
      prisma.audio.count({ where: { ...where, status: AudioStatus.PENDING } }),
      prisma.audio.count({ where: { ...where, status: AudioStatus.PROCESSING } }),
      prisma.audio.count({ where: { ...where, status: AudioStatus.DONE } }),
      prisma.audio.count({ where: { ...where, status: AudioStatus.ERROR } }),
    ]);

    const queueMetrics = await queueService.getQueueMetrics();

    res.json({
      success: true,
      data: {
        audios: { total, pending, processing, done, error },
        queue: queueMetrics,
      },
    });
  } catch (error) {
    next(error);
  }
 }

/**
 * Descarga el archivo Word de transcripción directamente
 */
async downloadTranscription(
  req: AuthRequest,
  res: Response,
  next: NextFunction
): Promise<void> {
  try {
    const { id } = req.params;
    const userId = req.user!.id;
    const isAdmin = req.user!.role === 'admin';

    // Buscar el audio
    const audio = await prisma.audio.findUnique({
      where: { id },
    });

    // Validaciones
    if (!audio) {
      throw new AppError('Audio no encontrado', StatusCodes.NOT_FOUND);
    }

    if (!isAdmin && audio.userId !== userId) {
      throw new AppError(
        'No tienes permiso para descargar este archivo',
        StatusCodes.FORBIDDEN
      );
    }

    if (!audio.driveTranscriptionFileId) {
      throw new AppError(
        'El archivo de transcripción no está disponible',
        StatusCodes.NOT_FOUND
      );
    }

    logger.info('Descargando transcripción desde Drive', {
      audioId: id,
      fileId: audio.driveTranscriptionFileId,
    });

    // Descargar archivo de Drive
    const fileBuffer = await driveService.downloadFileBuffer(
      audio.driveTranscriptionFileId
    );

    // Preparar nombre del archivo
    const filename = `${audio.originalFilename.replace(/\.[^/.]+$/, '')}_transcription.docx`;

    // Configurar headers para descarga
    res.setHeader(
      'Content-Type',
      'application/vnd.openxmlformats-officedocument.wordprocessingml.document'
    );
    res.setHeader(
      'Content-Disposition',
      `attachment; filename="${encodeURIComponent(filename)}"`
    );
    res.setHeader('Content-Length', fileBuffer.length);

    // Enviar el archivo
    res.send(fileBuffer);

    logger.info('Transcripción descargada exitosamente', {
      audioId: id,
      filename,
    });
  } catch (error: any) {
    logger.error('Error al descargar transcripción:', {
      audioId: req.params.id,
      error: error.message,
    });
    next(error);
  }
 }
}
export default new AudioController();
```